

4.2 数据库安全性控制

在计算机系统中,安全措施是一级一级层层设置的。例如,在图 4.1 所示的安全模型中,用户要求进入计算机系统时,系统首先根据输入的用户标识进行用户身份鉴别,只有合法的用户才准许进入计算机系统。对已进入系统的用户,数据库管理系统要执行安全保护,采取多种存取控制机制,目的是只允许用户执行合法操作,在用户退出系统后,根据情况还要进行安全审计。操作系统也会有自己的保护措施,读者可参考操作系统的有关书籍。对敏感数据,在传输过程中要进行加密保护,最后还应以密码形式存储到数据库中。



图 4.1 计算机系统的安全模型

本章讨论与数据库有关的安全性技术,对于强力逼迫透露口令、盗窃物理存储设备等行为而采取的保安措施,例如出入机房登记、加锁等,不在讨论之列。

图 4.2 是数据库管理系统的安全性控制模型示例。主要包含以下几部分:

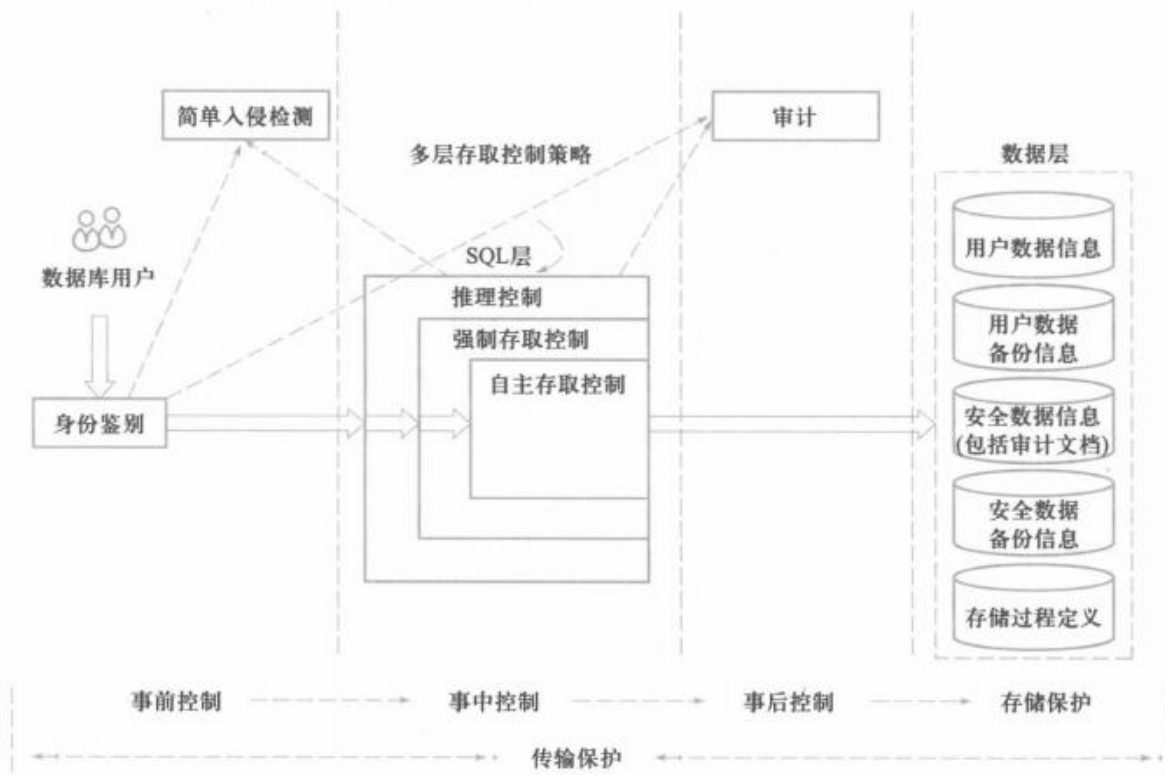


图 4.2 数据库管理系统的安全性控制模型

① 事前控制。通过身份鉴别和简单入侵检测共同实现事前的访问保护。

② 事中控制。数据库管理系统在 SQL 层提供多层存取控制策略,包括自主存取控制、强制存取控制、推理控制等。用户只能按照被授予的权限和安全规则存取合法数据。

③ 事后控制。为监控恶意访问,可根据具体安全需求配置审计规则,对用户访问行为和系统关键操作进行审计,及时发现可疑攻击者,把有关信息返回入侵检测子系统,对异常用户行为进行处理和阻拦。

④ 存储保护。在数据层,对敏感数据、重要的存储过程定义加密存储。数据库管理系统不仅存放用户数据,还存储与安全有关的审计文档,并进行加密存储。

⑤ 传输保护。用户身份信息、SQL 语句和数据等频繁地在客户端与服务器端进行传输,若采用明文方式传输很容易被截获或篡改,存在安全隐患。为了保证这些信息能够安全地进行交换和传输,数据库管理系统提供了传输加密功能。

4.2.1 用户身份鉴别

用户身份鉴别是数据库管理系统提供的最外层安全保护措施。每个用户在系统中都有一个用户标识。每个用户标识由用户名(user name)和用户标识号(user identification number, UID)两部分组成。UID 在系统的整个生命周期内是唯一的。系统内部记录着所有合法用户的标识,系统鉴别是指由系统提供一定的方式让用户标识自己的名字或身份。每次用户要求进入系统时,由系统进行核对,通过鉴定后才提供使用数据库管理系统的权限。

用户身份鉴别的方法有很多种,而且在一个系统中往往也是多种方法结合,以获得更强的安全性。常用的用户身份鉴别方法主要有以下几种。

1. 静态口令鉴别

静态口令一般由用户自己设定,鉴别时只要按要求输入正确的密码,系统即可允许用户使用数据库管理系统。这些密码是静态不变的,方式简单,但容易被攻击,安全性较低。因此通常采用双因子鉴别,即口令+数字证书。提供一套口令策略,包括密码复杂度、密码过期时限、登录失败用户锁定、密码历史保存等。在存储和传输过程中密码信息不可见,均以密文方式存在。

用户身份鉴别可以重复多次。

2. 动态口令鉴别

这种方式的口令是动态变化的,每次鉴别时均需使用动态产生的新口令登录数据库管理系统,即采用一次一密的方法。常用的方式如验证码和动态令牌方式,每次鉴别时要求用户使用验证码或动态令牌登录数据库管理系统。与静态口令鉴别相比,这种认证方式的安全性相对高一些。

3. 生物特征鉴别

它是一种通过生物特征进行认证的技术,其中,生物特征是指生物唯一具有的可测量、识别和验证的稳定生物特征,如指纹、虹膜、掌纹和人脸识别等。这种方式通过采用图像处理和

模式识别等技术实现了基于生物特征的认证,与传统的口令鉴别相比,其安全性无疑较高。

4. 智能卡鉴别

智能卡是一种不可复制的硬件,内置集成电路的芯片,具有硬件加密功能。智能卡由用户随身携带,登录数据库管理系统时将智能卡插入专用的读卡器进行身份验证。由于每次从智能卡中读取的数据是静态的,通过内存扫描或网络监听等技术还是可能截取到用户的身份验证信息,存在安全隐患。因此,实际应用中一般采用个人识别号(personal identification number, PIN)和智能卡相结合的方式。这样,即使PIN或智能卡中有一种被窃取,用户身份仍不会被冒充。

5. 入侵检测

检测前,管理员应按实际需求定义检测规则。在检测过程中,依据预先设置的检测规则实时进行入侵分析。若发现入侵情况,实时进行处理。

处理方式包括通过邮件报警和断开会话连接、锁定用户进行处罚等。

4.2.2 存取控制

数据库安全控制最重要的一点就是,确保只授权给有资格的用户访问数据库的权限,同时令所有未被授权的人员无法接近数据。这主要通过数据库系统的存取控制机制实现。

存取控制机制主要包括定义用户权限和合法权限检查两部分。

① 定义用户权限,并将用户权限登记到数据字典中。用户对某一数据对象的操作权力称为权限。某个用户应该具有何种权限是管理和政策方面的问题,而不是技术问题。数据库管理系统的功能是保证这些决定的执行。为此,数据库管理系统必须提供适当的语言来定义用户权限,这些定义经过编译后存储在数据字典中,被称作安全规则或授权规则。

② 合法权限检查。每当用户发出存取数据库的操作请求后(请求一般应包括操作类型、操作对象和操作用户等信息),数据库管理系统查找数据字典,根据安全规则进行合法权限检查,若用户的操作请求超出了定义的权限,系统将拒绝执行此操作。

定义用户权限和合法权限检查机制一起组成了数据库管理系统的存取控制子系统。

C2级的数据库管理系统支持自主存取控制,B1级的数据库管理系统支持强制存取控制。

这两类方法的简单定义是:

① 在自主存取控制方法中,用户对于不同的数据库对象有不同的存取权限,不同的用户对同一对象也有不同的权限,而且用户还可将其拥有的存取权限转授给其他用户。因此自主存取控制非常灵活。

② 在强制存取控制方法中,每一个数据库对象被标以一定的密级,每一个用户也被授予某一个级别的许可证。对于任意一个对象,只有具有合法许可证的用户才可以存取。强制存取控制因此相对比较严格。

下面介绍这两种存取控制方法。

4.2.3 自主存取控制方法

大型数据库管理系统都支持自主存取控制, SQL 标准也对自主存取控制提供支持。这主要通过 SQL 的 GRANT 语句和 REVOKE 语句来实现。

用户权限是由两个要素组成的: 数据库对象和操作类型。定义一个用户的存取权限就是要定义这个用户可以在哪些数据库对象上进行哪些类型的操作。在数据库系统中, 定义存取权限称为授权(authorization)。

在非关系数据库系统中, 用户只能对数据进行操作, 存取控制的数据库对象也仅限于数据本身。

在关系数据库系统中, 存取控制的对象不仅有数据本身(基本表和视图中的数据、属性列上的数据), 还有数据库与模式、基本表、视图和索引的创建等。表 4.4 列出了不同对象具有的主要存取权限。

表 4.4 关系数据库系统中不同对象具有的主要存取权限

对象类型	对象	操作类型
数据库和模式	数据库和模式	CREATE
	基本表	CREATE TABLE, ALTER TABLE
	视图	CREATE VIEW
	索引	CREATE INDEX
数据	基本表和视图	SELECT, INSERT, UPDATE, DELETE, REFERENCES, ALL PRIVILEGES
	属性列	SELECT, INSERT, UPDATE, REFERENCES, ALL PRIVILEGES

表 4.4 中, 创建数据库的权限在 SQL 标准中没有给出明确定义, 因此不同的关系数据库管理系统实现的语句和语法不同。4.2.4 小节将介绍 KingbaseES 创建数据库的语句。

具有创建数据库权限的用户可以在该数据库上创建模式。

具有创建模式权限的用户可以在该模式中创建表、视图、索引等数据库对象。

表 4.4 中, 属性列权限包括 SELECT、INSERT、UPDATE、REFERENCES, 其含义与表权限类似。需要说明的是, 对列的 UPDATE 权限指对于表中存在的某一列的值可以进行修改。当然, 有了这个权限之后, 在修改的过程中还要遵守表在创建时定义的主码及其他约束。列上的 INSERT 权限指用户可以插入一个元组, 对于插入的元组, 授权用户可以插入指定的值, 其他列或者为空, 或者为默认值。在给用户授予列 INSERT 权限时, 一定要包含主码的 INSERT 权限, 否则用户的插入动作会因为主码为空而被拒绝。

4.2.4 授予与收回对数据的操作权限

SQL 中使用 GRANT 和 REVOKE 语句向用户授予或收回对数据的操作权限。GRANT 语句向用户授予权限, REVOKE 语句收回已经授予用户的权限。

1. GRANT 语句

GRANT 语句的一般格式如下:

```
GRANT <权限> [, <权限>] ...  
ON <对象类型> <对象名> [, <对象类型> <对象名>] ...  
TO <用户> [, <用户>] ...  
[WITH GRANT OPTION];
```

其语义为: 将对指定对象的指定操作权限授予指定的用户。发出该 GRANT 语句的可以是数据库管理员 (DBA), 也可以是该数据库的创建者 (即数据库属主 owner), 还可以是已经拥有该权限的用户。接受权限的用户可以是一个或多个具体用户, 也可以是 PUBLIC, 即全体用户。

如果指定了 **WITH GRANT OPTION** 子句, 则获得某种权限的用户还可以把这种权限再授予其他的用户。如果没有指定该子句, 则获得某种权限的用户只能使用该权限, 不能传播。

SQL 标准允许具有 WITH GRANT OPTION 的用户把相应权限或其子集传递授予其他用户, 但不允许循环授权, 即被授权者不能把权限再授回给授权者或其祖先, 如图 4.3 所示。



图 4.3 SQL 标准不允许循环授权

[例 4.1] 把查询 Student 表的权限授给用户 U1。

```
GRANT SELECT  
ON TABLE Student  
TO U1;
```

[例 4.2] 把对 Student 表和 Course 表的全部操作权限授予用户 U2 和 U3。

```
GRANT ALL PRIVILEGES  
ON TABLE Student, Course  
TO U2, U3;
```

[例 4.3] 把对表 SC 的查询权限授予所有用户。

```
GRANT SELECT
```

```
ON TABLE SC  
TO PUBLIC;
```

[例 4.4] 把查询 Student 表和修改学生学号的权限授予用户 U4。

```
GRANT UPDATE(Sno),SELECT  
ON TABLE Student  
TO U4;
```

这里, 实际上要授予 U4 用户的是对基本表 Student 的 SELECT 权限和对属性列 Sno 的 UPDATE 权限。对属性列授权时必须明确指出相应的属性列名。

[例 4.5] 把对表 SC 的 INSERT 权限授予 U5 用户, 并允许将此权限再授予其他用户。

```
GRANT INSERT  
ON TABLE SC  
TO U5  
WITH GRANT OPTION;
```

执行此 SQL 语句后, U5 不仅拥有了对表 SC 的 INSERT 权限, 还可以传播此权限, 即由 U5 用户发上述 GRANT 命令给其他用户。

[例 4.6] 用户 U5 将对表 SC 的 INSERT 权限授予用户 U6。

```
GRANT INSERT  
ON TABLE SC  
TO U6  
WITH GRANT OPTION;
```

同样, U6 还可以将此权限继续授予 U7。

[例 4.7] 用户 U6 将对表 SC 的 INSERT 权限授予用户 U7。

```
GRANT INSERT  
ON TABLE SC  
TO U7;
```

因为 U6 未给 U7 传播的权限, 因此 U7 不能再传播此权限。

由上面的例子可以看到, GRANT 语句可以一次向一个用户授权, 如例 4.1 所示, 这是最简单的一种授权操作; 也可以一次向多个用户授权, 如例 4.2、4.3 所示; 还可以一次传播多个同类对象的权限, 如例 4.2 所示; 甚至一次可以完成对基本表和属性列这些不同对象的授权, 如例 4.4 所示。表 4.5 是执行了例 4.1~4.7 的语句后“学生选课”数据库中的用户权限定义表。

表 4.5 执行了例 4.1~4.7 语句后“学生选课”数据库中的用户权限定义

授权用户名	被授权用户名	数据库对象名	允许的操作类型	能否转授权
DBA	U1	关系 Student	SELECT	不能
DBA	U2	关系 Student	ALL	不能
DBA	U2	关系 Course	ALL	不能
DBA	U3	关系 Student	ALL	不能
DBA	U3	关系 Course	ALL	不能
DBA	PUBLIC	关系 SC	SELECT	不能
DBA	U4	关系 Student	SELECT	不能
DBA	U4	属性列 Student. Sno	UPDATE	不能
DBA	U5	关系 SC	INSERT	能
U5	U6	关系 SC	INSERT	能
U6	U7	关系 SC	INSERT	不能

2. REVOKE 语句

授予用户的权限可以由数据库管理员或其他授权者用 REVOKE 语句收回。REVOKE 语句的一般格式如下：

```
REVOKE <权限> [, <权限>] ...
ON <对象类型> <对象名> [, <对象类型> <对象名>] ...
FROM <用户> [, <用户>] ... [ CASCADE | RESTRICT ] ;
```

若语句指定了 CASCADE，则级联收回授予的权限；若语句指定了 RESTRICT，则转授权限后不能收回。默认值为 RESTRICT。

[例 4.8] 把用户 U4 修改学生学号的权限收回。

```
REVOKE UPDATE(Sno)
ON TABLE Student
FROM U4;
```

[例 4.9] 收回所有用户对表 SC 的查询权限。

```
REVOKE SELECT
ON TABLE SC
FROM PUBLIC;
```

[例 4.10] 把用户 U5 对 SC 表的 INSERT 权限收回。

```
REVOKE INSERT
```

```
ON TABLE SC
FROM U5 CASCADE;
```

执行该语句后，收回了用户 U5 的 INSERT 权限，同时级联收回了 U6 和 U7 的 INSERT 权限。因为在例 4.6 和例 4.7 中，U5 将对 SC 表的 INSERT 权限授予了 U6，而 U6 又将其授予了 U7。

注意：如果 U6 或 U7 还从其他用户处获得了对 SC 表的 INSERT 权限，则他们仍具有此权限，系统只收回直接或间接从 U5 处获得的权限。

表 4.6 是执行了例 4.8~4.10 的语句后“学生选课”数据库的用户权限定义。

表 4.6 执行了例 4.8~4.10 语句后“学生选课”数据库的用户权限定义

授权用户名	被授权用户名	数据库对象名	允许的操作类型	能否转授权
DBA	U1	关系 Student	SELECT	不能
DBA	U2	关系 Student	ALL	不能
DBA	U2	关系 Course	ALL	不能
DBA	U3	关系 Student	ALL	不能
DBA	U3	关系 Course	ALL	不能
DBA	U4	关系 Student	SELECT	不能

SQL 提供了非常灵活的授权机制。数据库管理员拥有对数据库中所有对象的所有权限，并可以根据实际情况将不同的权限授予不同的用户。

用户对自己建立的基本表和视图拥有全部的操作权限，并且可以用 GRANT 语句把其中某些权限授予其他用户。被授权的用户如果获得 WITH GRANT OPTION，还可以把获得的权限再授予其他用户。

所有授予出去的权力在必要时又都可以用 REVOKE 语句收回。

可见，用户可以“自主”地决定将数据的存取权限授予何人，并决定是否也将“授权”的权限授予别人。因此，称这样的存取控制是自主存取控制。

3. 创建数据库的权限

上面介绍的 GRANT 和 REVOKE 语句是向用户授予或收回对数据库中数据的操作权限。这里简单介绍一下表 4.4 中关于创建数据库的语句。因为创建数据库也需要进行安全控制，也是要授权的。但是在 SQL 标准中没有创建数据库的标准定义，因此各个数据库系统实现的语句不同。下面仅以金仓数据库 KingbaseES 为例做简要说明。

CREATE USER 语句建立超级用户，再由超级用户创建具有 CREATE 数据库权限的用户。

CREATE USER 语句一般语法格式如下：

```
CREATE USER <username> [ WITH ]
[ SUPERUSER | CREATEDB ] [ PASSWORD 'password' ];
```

具有 SUPERUSER 权限的用户是系统的超级用户，在系统中跳过权限检查，可以执行任何操作。

具有 CREATEDB 权限的用户可以创建数据库，成为数据库的属主，具有在数据库上创建模式的权限，并可以把这些权限授予其他用户。

注意，超级用户是系统初始化时指定的。例如，在安装系统时可以指定一个名称为 system 的超级用户。

[例 4.11] 创建超级用户 system2。

首先以超级用户 system 登录，然后创建 system2：

```
CREATE USER system2 WITH SUPERUSER PASSWORD '123456';
```

[例 4.12] 创建具有 CREATEDB 权限的用户 U1 和普通用户 U2。

以超级用户 system 登录，创建用户 U1、U2 如下：

```
CREATE USER U1 WITH CREATEDB PASSWORD '123456';  
/* U1 具有创建数据库的权限了 */  
CREATE USER U2 PASSWORD '123456';      /* U2 是普通用户 */
```

[例 4.13] 创建数据库 U1DB。

以 U1 用户登录，创建数据库 U1DB：

```
CREATE DATABASE U1DB;      /* U1 创建了数据库 */
```

U1 成为数据库 U1DB 的属主，它可以在 U1DB 数据库上创建 SCHEMA。

4.2.5 数据库角色

数据库角色是被命名的一组与数据库操作相关的权限，角色是权限的集合。因此，可以为具有相同权限的用户创建一个角色。使用角色来管理数据库权限可以简化授权的过程。

在 SQL 中首先用 CREATE ROLE 语句创建角色，然后用 GRANT 语句给角色授权，用 REVOKE 语句收回授予角色的权限。

1. 角色的创建

创建角色的 SQL 语句格式如下：

```
CREATE ROLE <角色名>
```

刚刚创建的角色是空的，没有任何内容。可以用 GRANT 为角色授权。

2. 给角色授权

```
GRANT <权限> [, <权限>] ...  
ON <对象类型> 对象名  
TO <角色> [, <角色>] ...
```

数据库管理员和用户可以利用 GRANT 语句将权限授予某一个或几个角色。

3. 将一个角色授予其他的角色或用户

```
GRANT <角色 1> [, <角色 2>] ...  
TO <角色 3> [, <用户 1>] ...  
[WITH ADMIN OPTION]
```

该语句把角色授予某用户,或授予另一个角色。这样,一个角色(例如角色 3)所拥有的权限就是授予它的全部角色(例如角色 1 和角色 2)所包含的权限的总和。

授予者可以是角色的创建者,或者是在这个角色上的 WITH ADMIN OPTION 权限拥有者。

如果指定了 WITH ADMIN OPTION 子句,则获得某种权限的角色或用户还可以把这种权限再授予其他的角色。

一个角色包含的权限包括直接授予这个角色的全部权限,再加上其他角色授予这个角色的全部权限。

4. 角色权限的收回

```
REVOKE <权限> [, <权限>] ...  
ON <对象类型> <对象名>  
FROM <角色> [, <角色>] ...
```

用户可以收回角色的权限,从而修改角色拥有的权限。

REVOKE 动作的执行者可以是角色的创建者,或者是在这个(些)角色上的 WITH ADMIN OPTION 权限拥有者。

[例 4.14] 通过角色来实现将一组权限授予一个用户。

步骤如下:

① 创建一个角色 R1。

```
CREATE ROLE R1;
```

② 使用 GRANT 语句,使角色 R1 拥有 Student 表的 SELECT、UPDATE、INSERT 权限。

```
GRANT SELECT, UPDATE, INSERT  
ON TABLE Student  
TO R1;
```

③ 将这个角色授予王平、张明、赵玲,使他们具有角色 R1 所包含的全部权限。

```
GRANT R1  
TO 王平, 张明, 赵玲;
```

④ 一次性通过 R1 来收回王平的这三个权限。

```
REVOKE R1  
FROM 王平;
```

[例 4.15] 给角色增加新的权限。

```
GRANT DELETE  
ON TABLE Student  
TO R1
```

使角色 R1 在原来的基础上增加了 Student 表的 DELETE 权限。

[例 4.16] 减少角色的权限。

```
REVOKE SELECT  
ON TABLE Student  
FROM R1;
```

使 R1 减少了 SELECT 权限。

可以看出, 数据库角色是一组权限的集合。使用角色来管理数据库权限可以简化授权的过程, 使自主授权的执行更加灵活、方便。

4.2.6 强制存取控制方法

自主存取控制能够通过授权机制有效地控制对敏感数据的存取。但是由于用户对数据的存取权限是“自主”的, 用户可以自由地决定将数据的存取权限授予何人, 以及决定是否也将“授权”的权限授予别人。在这种授权机制下, 仍可能存在数据的无意泄露。比如, 甲将自己权限范围内的某些数据存取权限授权给乙, 甲的意图是仅允许乙本人操纵这些数据。但甲的这种安全性要求并不能得到保证, 因为乙一旦获得了对数据的权限, 就可以将数据备份, 获得自身权限内的副本, 并在不征得甲同意的前提下传播副本。造成这一问题的根本原因就在于这种机制仅仅通过对数据的存取权限来进行安全控制, 而数据本身并无安全性标记。要解决这一问题, 就需要对系统控制下的所有主客体实施强制存取控制策略。

所谓强制存取控制, 是指系统为保证更高层次的安全性, 按照 TCSEC/TDI 标准中安全策略的要求所采取的强制存取检查手段。它不是用户能直接感知或进行控制的。强制存取控制适用于那些对数据严格且固定密级分类的部门, 例如军事部门或政府部门。

在强制存取控制方法中, 数据库管理系统所管理的全部实体被分为主体和客体两大类。

主体是系统中的活动实体, 既包括数据库管理系统所管理的实际用户, 也包括代表用户的各进程。**客体**是系统中的被动实体, 是受主体操纵的, 包括文件、基本表、索引、视图等。对于主体和客体, 数据库管理系统为它们的每个实例(值)指派一个敏感度标记(label)。

敏感度标记被分成若干级别, 例如绝密级(top secret, TS)、机密级(secret, S)、秘密级(confidential, C)、公开(public, P)等。密级的次序是 $TS \geq S \geq C \geq P$ 。主体的敏感度标记称为许可证级别, 客体的敏感度标记称为密级。强制存取控制机制就是通过对比主体和客体

的 label, 最终确定主体是否能够存取客体。

当某一用户(或主体)以标记 label 注册入系统时, 系统要求他对任何客体的存取必须遵循如下规则:

- ① 仅当主体的许可证级别大于或等于客体的密级时, 该主体才能读取相应的客体。
- ② 仅当主体的许可证级别小于或等于客体的密级时, 该主体才能写相应的客体。

这些系统规定: 仅当主体的许可证级别小于或等于客体的密级时, 该主体才能写相应的客体, 即用户可以为写入的数据对象赋予高于自己许可证级别的密级。这样一旦数据被写入, 该用户自己也不能再读该数据对象了。

规则①的含义是明显的, 而规则②需要解释一下。按照规则②, 用户可以为写入的数据对象赋予高于自己的许可证级别的密级。这样一旦数据被写入, 该用户自己也不能再读该数据对象了。如果违反了规则②, 就有可能把数据的密级从高流向低, 造成数据的泄漏。

例如, 某个绝密级的用户把一个绝密级别的数据恶意地降低为公开级别, 然后把它写回, 这样原来是绝密的数据大家都可以读到了, 造成了绝密级数据的泄漏。

这两种规则的共同点在于, 它们均禁止了拥有高许可证级别的主体更新低密级的数据对象, 从而防止了敏感数据的泄漏。

强制存取控制是对数据本身进行密级标记, 无论数据如何复制, 标记与数据是一个不可分的整体。只有符合密级标记要求的用户才可以操纵数据, 从而提供了更高级别的安全性。

前面已经提到, 较高安全性级别提供的安全保护包含较低级别的所有保护, 因此在实现强制存取控制时要首先实现自主存取控制, 即自主存取控制与强制存取控制共同构成数据库管理系统的安全机制。图 4.4 为数据库管理系统的安全检查示意图, 系统首先进行自主存取控制(DAC)检查, 对通过 DAC 检查的允许存取的数据对象, 再由系统自动进行强制存取控制(MAC)检查, 只有通过 MAC 检查的数据对象方可存取。



图 4.4 数据库管理系统的安全检查示意图

4.3 视图机制

前已提及, 视图具有安全保护的功能。为不同的用户定义不同的视图, 就可以把数据对象限制在一定的范围内, 通过视图机制把保密的数据对无权存取的用户隐藏起来, 从而自动对数据提供一定程度的安全保护。

视图机制还可以和授权机制相配合, 间接地实现支持存取谓词的用户权限定义。例如, 假定王平老师只能查询计算机科学与技术专业学生的信息, 专业负责人张明老师具有查询和增删改该专业学生信息的所有权限。这就要求系统能支持存取谓词的用户权限定义。在不直接支持存取谓词的系统中, 可以先建立计算机科学与技术专业学生的视图 CS_Student, 然后在视图上